

Curso desarrollo de aplicaciones con tecnologías web
Módulo formativo 2: Programación web en el entorno servidor

Unidad formativa 2: Acceso a datos en aplicaciones web del entorno servidor

Tema 5: JSON: Lenguaje de Marcas para el Intercambio de Datos

Origen e Historia de JSON

Características de JSON

Estructura de JSON

Estándares basados en JSON

Análisis de JSON

Uso de JSON en el Intercambio de Información

Origen e Historia de JSON

JSON (JavaScript Object Notation) es un formato de intercambio de datos ligero y de fácil lectura, creado por Douglas Crockford en 2001. Se originó como una alternativa más sencilla a XML para la transmisión de datos entre cliente y servidor en aplicaciones web.

Características de JSON

JSON es un formato basado en texto que representa estructuras de datos como objetos y arrays. Sus características principales son:

- Ligero y fácil de leer.
- Basado en una estructura de pares clave-valor.
- Se integra nativamente en JavaScript y es compatible con muchos lenguajes.
- No requiere etiquetas de apertura y cierre como XML.

Ejemplo de un objeto JSON:

```
json
{
  "nombre": "Juan",
  "edad": 30,
  "ciudad": "Madrid"
}
```

Estructura de JSON

1 Elementos básicos de JSON

- **Objetos:** Conjunto de pares clave-valor encerrados entre llaves `{}`.
- **Arrays:** Lista de valores encerrados entre `[]`.
- **Valores:** Cadenas, números, booleanos, `null`, objetos o arrays.

Ejemplo con un array dentro de un objeto JSON:

```
json
{
  "nombre": "Ana",
  "edad": 25,
  "aficiones": ["leer", "viajar", "fotografía"]
}
```

Estándares basados en JSON

1 JSON Schema

Similar a XML Schema, JSON Schema permite definir la estructura y validación de documentos JSON.

Ejemplo de JSON Schema:

```
json
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "properties": {
    "nombre": { "type": "string" },
    "edad": { "type": "integer" }
  },
  "required": ["nombre", "edad"]
}
```

Análisis de JSON

1 Herramientas de análisis

Algunas herramientas para trabajar con JSON incluyen:

- **jq** (línea de comandos para analizar JSON)
- **JSONLint** (validador online de JSON)
- **Extensiones para navegadores** (como JSONView en Chrome y Firefox)

2 Programación de análisis JSON en PHP

En PHP, se pueden utilizar las funciones `json_encode()` y `json_decode()` para trabajar con JSON.

Ejemplo: Convertir un array PHP en JSON

```
php
$datos = [
    "nombre" => "Pedro",
    "edad" => 40
];
$json = json_encode($datos);
echo $json;
```

Ejemplo: Convertir JSON a un array PHP

```
php
$json = '{"nombre": "Pedro", "edad": 40}';
$datos = json_decode($json, true);
echo $datos["nombre"]; // Pedro
```

Uso de JSON en el Intercambio de Información

1 JSON y APIs

La mayoría de las APIs REST utilizan JSON para enviar y recibir datos. Ejemplo de una respuesta JSON en una API:

```
json
{
  "usuarios": [
    { "id": 1, "nombre": "Laura" },
    { "id": 2, "nombre": "Carlos" }
  ]
}
```

2 Uso de JSON en Configuración

Al igual que XML, JSON se usa para almacenar configuraciones.

Ejemplo de un archivo `config.json`:

```
json
{
  "base_datos": {
    "host": "localhost",
    "usuario": "root",
    "password": "1234"
  }
}
```

Cómo leerlo en PHP:

```
php
$config = json_decode(file_get_contents("config.json"), true);
echo $config["base_datos"]["host"]; // localhost
```

3 JSON en Bases de Datos MySQL

MySQL permite almacenar y consultar datos en formato JSON mediante el tipo de dato `JSON`.

Crear una tabla con una columna JSON:

```
sql
CREATE TABLE productos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    datos JSON
);
```

Insertar datos JSON en MySQL:

```
sql
INSERT INTO productos (datos) VALUES ('{"nombre": "Laptop", "precio": 1200}');
```

Consultar datos JSON en MySQL:

```
sql
SELECT datos->'$.nombre' AS nombre FROM productos;
```

Conclusión

JSON es un formato de datos ligero y flexible que ha reemplazado a XML en muchas aplicaciones modernas. Su integración con PHP y MySQL lo convierte en una herramienta esencial para el desarrollo web actual.

Ejercicio: Guardar y recuperar datos en JSON desde MySQL

Descripción:

Vamos a crear una tabla en MySQL que almacene datos en formato JSON y usaremos PHP para insertar y recuperar estos datos.

1. Crear la tabla en MySQL

Ejecuta esta consulta en tu base de datos MySQL:

sql

```
CREATE TABLE productos (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    datos JSON NOT NULL  
);
```


2. Insertar datos en JSON

Creamos un script en PHP para insertar un producto con sus detalles en la base de datos.

php

```
<?php
// Conexión a la base de datos
$pdo = new PDO("mysql:host=localhost;dbname=tu_base_de_datos;charset=utf8", "usuario",
"contraseña");

// Datos en formato JSON
$producto = [
    "nombre" => "Laptop",
    "precio" => 1200.50,
    "stock" => 10
];

// Convertimos el array a JSON
$jsonProducto = json_encode($producto);

// Insertamos en la base de datos
$sql = "INSERT INTO productos (datos) VALUES (:datos)";
$stmt = $pdo->prepare($sql);
$stmt->bindParam(':datos', $jsonProducto);
$stmt->execute();

echo "Producto insertado correctamente.";
?>
```

3. Recuperar y mostrar datos desde JSON

Ahora, recuperaremos los productos desde la base de datos y los mostraremos en pantalla.

```
<?php
// Conexión a la base de datos
$pdo = new PDO("mysql:host=localhost;dbname=tu_base_de_datos;charset=utf8", "usuario",
"contraseña");

// Consulta de productos
$sql = "SELECT datos FROM productos";
$stmt = $pdo->query($sql);

// Mostramos los productos
while ($fila = $stmt->fetch(PDO::FETCH_ASSOC)) {
    $producto = json_decode($fila['datos'], true);
    echo "Nombre: " . $producto['nombre'] . "<br>";
    echo "Precio: " . $producto['precio'] . "€<br>";
    echo "Stock: " . $producto['stock'] . "<br><br>";
}
?>
```

Explicación:

1. Creamos la tabla con un campo JSON para almacenar los datos en este formato.
2. Insertamos datos en formato JSON, usando `json_encode()` para convertir un array asociativo en JSON.
3. Recuperamos y mostramos los datos, usando `json_decode()` para convertir el JSON en un array de PHP.